

2.4.1 Sorting Algorithms: Bubble Sort

سارٹنگ (Sorting) کا مطلب ہے ڈیٹا کو صعودی (چھوٹے سے بڑے) یا نزولی (بڑے سے چھوٹے) ترتیب میں رکھنا۔

CS XI

Chapter 2: Computational Thinking & Algorithm

Lecture 06 of 11

★ Golden

2.4.1 Sorting Algorithms: Bubble Sort ★ Golden

LEARN IT

Sorting and searching are both fundamental techniques of computer science. They work on **lists of the same data type** (for example, a list of all integers or a list of all names).

Sorting	Searching
Arranging a collection of items in an order. The order may be ascending (0–9, A–Z) or descending (9–0, Z–A). Used to organise data.	Locating a specific element within a list to find its position or check its existence (is it there or not?).
Examples: Bubble Sort, Selection Sort	Examples: Linear Search, Binary Search
Real life: merit list arranged from highest to lowest marks.	Real life: finding your roll number in the merit list.

Notation you will see

- **DATA** — the list (array). `DATA[1]` is the 1st element in the textbook algorithms (counting starts at 1).
- In **pseudocode and Python** counting starts at **0**: `DATA[0]` is the 1st element. This is called an **index**.
- **N** — the number of elements in the list.
- **PTR** (pointer) — a variable that holds the current position.
- **temp** — a temporary variable used to **swap** (exchange) two values.
- **Pass** — one complete run of the inner loop over the list.

WORKED EXAMPLE

How a swap works with temp. Suppose $A = 8$ and $B = 4$. To exchange them:

`temp = A (temp = 8)` → `A = B (A = 4)` → `B = temp (B = 8)`

Without `temp`, writing `A = B` first would destroy the 8. In Python you may also write `a, b = b, a`.

COMMON MISTAKE

Position 1 in the textbook algorithm is index 0 in Python. In exams, if a question uses `DATA[1]`, count from 1; if it uses Python, count from 0. State clearly which one you use.

اردو میں سمجھیں

سارٹنگ (Sorting) کا مطلب ہے ڈیٹا کو صعودی (چھوٹے سے بڑے) یا نزولی (بڑے سے چھوٹے) ترتیب میں رکھنا۔ سرچنگ (Searching) کا مطلب ہے فہرست میں کسی خاص چیز کو تلاش کرنا تاکہ اس کی جگہ یا موجودگی معلوم ہو سکے۔ دونوں ایک ہی قسم کے ڈیٹا کی فہرست پر کام کرتے ہیں۔

CHECK YOURSELF

1. Arrange C, A, D, B in descending order.
2. What is the purpose of the variable temp?

Answers: 1. D, C, B, A. 2. It temporarily holds one value so that two values can be swapped without losing either.

LEARN IT

Bubble Sort is an elementary sorting algorithm that works by repeatedly comparing **adjacent** (side-by-side) elements and swapping them if they are in the wrong order.

In each pass, the largest remaining value "bubbles up" to the end of the list, like an air bubble rising to the top of water. So after pass 1 the largest value is in the last place, after pass 2 the second largest is in second-last place, and so on. A list of N items needs $N - 1$ **passes**, and pass K makes $N - K$ **comparisons**.

`Compare DATA[j] and DATA[j+1]` → `Left > Right?` → `Yes: swap` → `Move one step right` →
`End of pass: largest is at the end` → `Repeat N-1 passes`

Algorithm (textbook)

Pseudocode

```

Step 1: Start
Step 2: Set DATA := {8,4,1,9,3}
Step 3: Set N := 5
Step 4: Repeat Step 5 and 6 for
        K = 1 to N-1
Step 5: Set PTR := 1
Step 6: Repeat while PTR ≤ N-K:
    a: If DATA[PTR] > DATA[PTR+1]:
        1. Set temp = DATA[PTR]
        2. Set DATA[PTR] = DATA[PTR+1]
        3. Set DATA[PTR+1] = temp
    b: Set PTR := PTR + 1
Step 7: End

```

```

Start
DATA = {8,4,1,9,3}
N = 5
For outer = 0 to N-1
    For inner = 0 to N-2-outer
        IF DATA[inner] > DATA[inner+1]
            Temp = DATA[inner]
            DATA[inner] = DATA[inner+1]
            DATA[inner+1] = Temp
        End If
    End For
End For
End

```

WORKED EXAMPLE

Worked example: sort 8, 4, 1, 9, 3 in ascending order. The pair in brackets [] is being compared. Positions are counted from 1.

Pass 1 (K = 1, 4 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[8, 4], 1, 9, 3	8 > 4 Yes	Swap	4, 8, 1, 9, 3
2nd vs 3rd	4, [8, 1], 9, 3	8 > 1 Yes	Swap	4, 1, 8, 9, 3
3rd vs 4th	4, 1, [8, 9], 3	8 > 9 No	No change	4, 1, 8, 9, 3
4th vs 5th	4, 1, 8, [9, 3]	9 > 3 Yes	Swap	4, 1, 8, 3, 9

9 is now at its correct (final) place.

Pass 2 (K = 2, 3 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[4, 1], 8, 3, 9	4 > 1 Yes	Swap	1, 4, 8, 3, 9
2nd vs 3rd	1, [4, 8], 3, 9	4 > 8 No	No change	1, 4, 8, 3, 9
3rd vs 4th	1, 4, [8, 3], 9	8 > 3 Yes	Swap	1, 4, 3, 8, 9

Pass 3 (K = 3, 2 comparisons)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[1, 4], 3, 8, 9	1 > 4 No	No change	1, 4, 3, 8, 9
2nd vs 3rd	1, [4, 3], 8, 9	4 > 3 Yes	Swap	1, 3, 4, 8, 9

Pass 4 (K = 4, 1 comparison)

Comparison	Pair	Left > Right?	Action	List after
1st vs 2nd	[1, 3], 4, 8, 9	1 > 3 No	No change	1, 3, 4, 8, 9

Sorted list: 1, 3, 4, 8, 9. Totals: 4 + 3 + 2 + 1 = **10 comparisons** and **6 swaps**.

Pass	List after pass	Fixed at the end
Start	8, 4, 1, 9, 3	—
1	4, 1, 8, 3, 9	9
2	1, 4, 3, 8, 9	8, 9

3	1, 3, 4, 8, 9	4, 8, 9
4	1, 3, 4, 8, 9	all

WORKED EXAMPLE

Bubble Sort in Python (follows the pseudocode exactly; Python indexes start at 0):

```
data = [8, 4, 1, 9, 3]
n = len(data)
for outer in range(n - 1):
    for inner in range(n - 1 - outer):
        if data[inner] > data[inner + 1]:
            temp = data[inner]
            data[inner] = data[inner + 1]
            data[inner + 1] = temp
    print("After pass", outer + 1, ":", data)
print("Sorted list:", data)
```

OUTPUT

```
After pass 1 : [4, 1, 8, 3, 9]
After pass 2 : [1, 4, 3, 8, 9]
After pass 3 : [1, 3, 4, 8, 9]
After pass 4 : [1, 3, 4, 8, 9]
Sorted list: [1, 3, 4, 8, 9]
```

`range(n - 1 - outer)` gives `inner = 0 ... N-2-outer`, exactly as in the pseudocode. Notice the list was already sorted after pass 3, but the plain algorithm still does pass 4.

Advantages and disadvantages

Advantages	Disadvantages
Easiest algorithm to visualise and explain.	Uses nested loops, so the time grows very fast as the list grows (about N^2 comparisons).
Does not need extra memory (in-place sorting).	Performs a massive number of swaps.
Stable (equal items stay in their original order).	Useless for real-world "Big Data".

REMEMBER IT

Bubble = neighbours. Bubble Sort only ever compares *neighbours* (j and $j+1$). Each pass pushes the biggest bubble to the end. For **descending** order, just change `>` to `<` in the comparison.

COMMON MISTAKE

The slides say the time grows "exponentially". Strictly, Bubble Sort's time grows **quadratically**: for N items it makes $N(N-1)/2$ comparisons, which is $O(N^2)$. Doubling the list makes it about 4 times slower — bad, but not exponential. Another common mistake: students forget that pass K does only $N - K$ comparisons; the sorted tail is never compared again.

اردو میں سمجھیں

بل سارٹ (Bubble Sort) میں ساتھ ساتھ موجود دو عناصر کا موازنہ کیا جاتا ہے اور اگر وہ غلط ترتیب میں ہوں تو ان کی جگہ بدل دی جاتی ہے۔ ہر پاس (Pass) کے بعد سب سے بڑا عدد آخر میں پہنچ جاتا ہے۔ N عناصر کے لیے $N-1$ پاس درکار ہوتے ہیں۔ یہ سمجھنے میں آسان اور مستحکم (Stable) ہے لیکن بڑے ڈیٹا کے لیے بہت سست ہے۔

EXAM TIP

The most common question: "Sort the list ... using Bubble Sort. Show each pass." Draw one table per pass with columns *Pair*, *Compare*, *Action*, *List after*. Mark the element fixed at the end after every pass. Also be ready to write the algorithm (with PTR and K) or the pseudocode (with outer/inner), and to list two advantages and two disadvantages.

CHECK YOURSELF

1. How many passes and comparisons does Bubble Sort need for 6 elements?
2. Show the list 5, 1, 4 after pass 1 of Bubble Sort.
3. Sort 2, 9, 4, 7 in descending order with Bubble Sort: give the list after each pass.

Answers: 1. 5 passes; $5 + 4 + 3 + 2 + 1 = 15$ comparisons. 2. (5,1) swap $\rightarrow$ 1, 5, 4; (5,4) swap $\rightarrow$ 1, 4, 5. 3. Pass 1: 9, 4, 7, 2; Pass 2: 9, 7, 4, 2; Pass 3: 9, 7, 4, 2.

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. Bubble sort repeatedly:

- a. Swaps adjacent out-of-order pairs
- b. Picks the global min into place
- c. Halves a sorted list
- d. Hashes keys

MCQ 2. After pass 1 of bubble, the largest item is:

- a. At the end
- b. At the start always
- c. Deleted
- d. In a stack

MCQ 3. Best-case bubble (already sorted, with flag) is:

- a. $O(n)$
- b. $O(n^2)$ always
- c. $O(1)$
- d. $O(n!)$

MCQ 4. Typical/worst bubble is:

- a. $O(n^2)$
- b. $O(\log n)$
- c. $O(n)$
- d. $O(n^3)$

MCQ 5. On 8,4 the first swap gives:

- a. 4,8
- b. 8,4
- c. 0,0
- d. 4,4

MCQ 6. Bubble is easy but:

- a. Slow on large n
- b. Faster than binary search on sorted data
- c. A graph algorithm
- d. $O(1)$ memory sort of magic

Extra short questions

1. Define bubble sort.
2. Dry-run one pass on 8,4,1.
3. Space complexity?
4. When is bubble acceptable?

Extra long question

1. Trace bubble sort on 8,4,1,9,3 until sorted. Show each pass. State comparisons in the worst case for $n=5$.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Repeatedly compare adjacent items and swap if they are in the wrong order so large values "bubble" to the end.
2. $8 > 4$ swap $\rightarrow 4, 8, 1$; $8 > 1$ swap $\rightarrow 4, 1, 8$. Largest 8 is at the end.
3. $O(1)$ extra — in-place.
4. Tiny lists or teaching; not for thousands of records.

Long

1. Passes: after p1 ...3,8,4,1,9 wait — actually $8, 4, 1, 9, 3 \rightarrow 4, 1, 8, 3, 9 \rightarrow 1, 4, 3, 8, 9 \rightarrow 1, 3, 4, 8, 9 \rightarrow 1, 3, 4, 8, 9$. Worst comparisons $n(n-1)/2 = 10$.

Next lecture: 2.4.1 Sorting Algorithms: Selection Sort